

Encryption Method

1. Install Jenkins on your local machine or use a Jenkins Docker container, then access the Jenkins dashboard and take a screenshot of the home page to confirm your setup.

ANS :

- I installed Jenkins on my system and started the Jenkins service. After installation, I opened Jenkins in a web browser using:
 - `http://localhost:8080`
 - I completed the initial Jenkins setup and logged in to the Jenkins dashboard.
 - The Jenkins home page was displayed successfully, confirming that Jenkins was installed and running correctly.
2. Create a new freestyle Jenkins job named 'SpotifyPlaylistBuild' that simply echoes 'Building Spotify Playlist App' in the build step, then run the job and check the console output.

ANS :

- 1) Open Jenkins Dashboard.
- 2) Click **New Item**.
- 3) Enter the name:
 - SpotifyPlaylistBuild
- 4) Select **Freestyle project**.
- 5) Click **OK**.
- 6) Go to **Build Steps**.
- 7) Select **Execute shell**.
- 8) Enter:
 - `echo "Building Spotify Playlist App"`
- 9) Click **Save**.
- 10) Click **Build Now**.
- 11) Open **Console Output**.

❖ **Expected Output:**

- Building Spotify Playlist App
 - Finished: SUCCESS
- This confirms that the Jenkins Freestyle job executed successfully.

3. Set up a basic Jenkins pipeline (using Pipeline script) that checks out code from any public GitHub repository (for example, a simple JavaScript or Python app), then runs a build step using 'echo' to simulate a build process.

Hint: Use the 'pipeline' syntax with 'stages' for checkout and build.

ANS :

```
pipeline {
  agent any
  stages {

    stage('Checkout') {
      steps {
        git 'https://github.com/jenkins-docs/simple-java-maven-app.git'
      }
    }

    stage('Build') {
      steps {
        echo 'Building Spotify Playlist App'
      }
    }
  }
}
```

1. Open Jenkins Dashboard.
2. Click **New Item**.
3. Enter:
 - SpotifyPlaylistPipeline
4. Select **Pipeline**.
5. Click **OK**.
6. Scroll to the **Pipeline** section.
7. Select **Pipeline script**.
8. Paste the above script.
9. Click **Save**.
10. Click **Build Now**.
11. Open **Console Output**.

4. Modify your Jenkins pipeline to add a test stage that runs a simple test command (for example, 'npm test' for a Node.js app or 'python -m unittest' for Python), and configure the pipeline to fail if tests do not pass.

Hint: If you don't have real tests, use a dummy command like 'exit 1' to simulate a failure and see how Jenkins handles it.

ANS :

```
pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps {
        git 'https://github.com/jenkins-docs/simple-java-maven-app.git'
      }
    }
    stage('Build') {
      steps {
        echo 'Building Spotify Playlist App'
      }
    }
    stage('Test') {
      steps {
        sh 'mvn test'
      }
    }
  }
}
```

5. Use ChatGPT or GitHub Copilot to help you write a Jenkinsfile that builds and tests a Zomato-style food ordering app (can be a simple placeholder app), then copy and use this Jenkinsfile in a new pipeline job and describe any errors or improvements suggested by the AI tool.

ANS :

- 1) Adding automated testing so the application is tested before deployment.
- 2) Using environment variables instead of hard-coding configuration values.
- 3) Adding deployment credentials securely through Jenkins Credentials instead of putting passwords or keys inside the Jenkinsfile.
- 4) Adding notifications so the development team is informed when a build fails.
- 5) Adding Docker or Kubernetes deployment for a more realistic cloud-based application.

```

pipeline {
    agent any
    stages {
        stage('Checkout') {
            steps {
                echo 'Checking out Food Ordering App source code'

                git 'https://github.com/jenkins-docs/simple-java-maven-app.git'
            }
        }

        stage('Build') {
            steps {
                echo 'Building Zomato-style Food Ordering App'
                sh 'mvn clean package -DskipTests'
            }
        }

        stage('Test') {
            steps {
                echo 'Running application tests'

                sh 'mvn test'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying Food Ordering App'
                echo 'Deployment completed successfully'
            }
        }
    }

    post {
        success {
            echo 'Food Ordering App pipeline completed successfully.'
        }
        failure {
            echo 'Food Ordering App pipeline failed. Please check the console
output.'
        }
    }
}

```